

Bagus. Jika tujuanmu membuat challenge CTF, maka harus ada dua sisi:

1. **Author** → membuat challenge.
2. **Player** → menyelesaikan challenge.

Mari kita buat contoh lengkap untuk **LCG Prediction Attack**.

Challenge 1 — LCG Prediction (Easy)

Sebagai Author

Buat file challenge:

```
a = 1103515245
```

```
c = 12345
```

```
m = 2**31
```

```
seed = 1337
```

```
def rand():
```

```
    global seed
```

```
    seed = (a * seed + c) % m
```

```
    return seed
```

```
print(rand())
```

```
print(rand())
```

```
print(rand())
```

Jalankan:

```
78628734
```

```
1460962527
```

```
934458668
```

Sekarang hitung output berikutnya:

```
next_value = (1103515245 * 934458668 + 12345) % (2**31)
```

```
print(next_value)
```

Misal hasilnya:

1985133557

Flag:

CTF{1985133557}

File yang Diberikan ke Peserta

I've built a secure random number generator.

78628734

1460962527

934458668

Flag format:

CTF{next_output}

Cara Resolve

Peserta melihat source atau menebak bahwa generator adalah LCG.

Diketahui:

$a = 1103515245$

$c = 12345$

$m = 2^{*}31$

Output terakhir:

934458668

Hitung:

$a = 1103515245$

$c = 12345$

$m = 2^{*}31$

$x = 934458668$

$\text{print}((a*x+c)\%m)$

Output:

1985133557

Flag:

CTF{1985133557}

Challenge 2 — Recover Seed

Lebih menarik.

Sebagai Author

$a = 1664525$

$c = 1013904223$

$m = 2^{32}$

seed = 123456

```
def rand():
```

```
    global seed
```

```
    seed = (a*seed+c)%m
```

```
    return seed
```

```
print(rand())
```

Output:

3510724159

Flag:

CTF{123456}

Yang Diberikan

Generator:

$X_{n+1} = (1664525 \cdot X_n + 1013904223) \bmod 2^{32}$

First output:

3510724159

Recover the seed.

Cara Resolve

Karena:

$$X1 = (aX0 + c) \bmod m$$

Maka:

$$X0 = a^{-1}(X1 - c) \bmod m$$

Cari inverse:

$$a = 1664525$$

$$c = 1013904223$$

$$m = 2^{32}$$

$$x1 = 3510724159$$

$$a_{inv} = \text{pow}(a, -1, m)$$

$$\text{seed} = ((x1 - c) * a_{inv}) \% m$$

print(seed)

Output:

123456

Flag:

CTF{123456}

Challenge 3 — Recover Parameter (Medium)

Ini yang sering muncul.

Sebagai Author

Pilih:

a = 17

c = 43

m = 1009

seed = 123

Generate:

123

116

1006

994

Misalnya.

Flag:

CTF{17_43_1009}

Yang Diberikan

Observed outputs:

123

116

1006

994

Recover:

a

c

m

Cara Resolve

Gunakan:

x0

x1

x2

x_3

Hitung:

$$t_0 = x_1 - x_0$$

$$t_1 = x_2 - x_1$$

$$t_2 = x_3 - x_2$$

Lalu gunakan teknik:

gcd(

$$t_2 * t_0 - t_1^2,$$

...

)

untuk memperoleh modulus.

Setelah modulus diketahui:

$$a = t_1 * \text{inverse}(t_0) \bmod m$$

Kemudian:

$$c = x_1 - a * x_0 \bmod m$$

Challenge 4 — MT19937 Clone State (Medium)

Ini challenge yang sangat populer.

Sebagai Author

```
import random
```

```
random.seed(1337)
```

```
for _ in range(624):
```

```
    print(random.getrandbits(32))
```

```
flag = f'CTF{{{random.getrandbits(32)}}}'
```

Yang Diberikan

File:

Cara Resolve

Install:

```
pip install randcrack
```

Solver:

```
from randcrack import RandCrack
```

```
rc = RandCrack()
```

for value in outputs:

```
    rc.submit(value)
```

```
print(rc.predict_getrandbits(32))
```

Output yang diprediksi adalah isi flag.

Challenge 5 — Timestamp Seed (Easy)

Sebagai Author

```
import random,time
```

```
seed = int(time.time())
```

```
random.seed(seed)
```

```
print(random.getrandbits(32))
```

Misal:

1856145921

Flag:

CTF{1748934000}

(seed timestamp)

Yang Diberikan

Server started recently.

Output:

1856145921

Cara Resolve

```
import random,time
```

```
target = 1856145921
```

```
now = int(time.time())
```

```
for s in range(now-50000, now+1):
```

```
    random.seed(s)
```

```
    if random.getrandbits(32)==target:
```

```
        print(s)
```

```
        break
```

Challenge 6 — LFSR Stream Cipher (Hard)

Sebagai Author

```
flag = b"CTF{example}"
```

```
keystream = generate_lfsr(secret_state)
```

```
cipher = xor(flag, keystream)
```

```
print(cipher.hex())
```

Yang Diberikan

ciphertext:

6f1b4a8c...

Hint:

Flag starts with:

CTF{

Cara Resolve

Karena mengetahui plaintext awal:

CTF{

maka:

keystream = ciphertext XOR plaintext

Dapatkan beberapa bit keystream.

Kemudian gunakan algoritma Berlekamp–Massey untuk mencari:

- panjang register
- feedback polynomial
- state

Setelah state ditemukan:

generate_remaining_keystream()

decrypt()

Urutan Belajar yang Disarankan

1. LCG Prediction ★
2. Recover Seed ★ ★
3. Recover Parameter ★ ★ ★
4. MT19937 Timestamp ★ ★ ★
5. MT19937 Clone State ★ ★ ★ ★
6. LFSR + Berlekamp-Massey ★ ★ ★ ★ ★

Jika ingin membuat challenge yang realistis seperti di CTF modern, setelah LCG saya sarankan langsung belajar **MT19937 Clone State** karena itu salah satu attack PRNG yang paling sering muncul dan cukup elegan untuk dijadikan soal Jeopardy.

